

PROJECT_SPEC.md: Predictive Mutation-Based Fault Localization (ML-PMT)

1. Project Overview

This project implements a Predictive Mutation-Based Fault Localization (ML-PMT) pipeline designed specifically for mathematical and Machine Learning architectures. Traditional mutation testing is too computationally expensive for ML pipelines because it requires full test suite execution for every generated mutant.

This architecture bypasses the execution bottleneck by using a lightweight machine learning classifier to predict mutant survival based on structural AST features. The predicted outcomes are fed into a standard MBFL formula to generate a ranked list of suspicious lines, allowing for rapid debugging of silent logical errors (e.g., matrix shape mismatches) in models like Physics-Informed Neural Networks (PINNs) or Markowitz Mean-Variance Optimization scripts.

2. System Architecture & Core Modules

- **Target Subject & Oracle:** A deterministic Python mathematical pipeline (e.g., a Markowitz portfolio optimizer) validated by a strict pytest suite containing assertions for mathematical constraints (e.g., `weights.sum() == 1.0`, matrix dimensions).
- **AST Mutation Engine:** A custom syntax transformer built on LibCST that parses the target Python source code and systematically injects domain-specific bugs without losing formatting.
- **Ground Truth Execution Runner:** A controller that executes pytest via subprocess against every generated mutant to record actual 1 (Killed) or 0 (Survived) labels for training.
- **Feature Extraction Pipeline:** An analyzer that extracts syntax and execution metadata (mutation operator type, AST nesting depth, parent node context, and test coverage).
- **Predictive Classifier & Ranker:** A `RandomForestClassifier` that predicts mutant survival probabilities. These predictions are passed to the Metallaxis formula to rank line-level suspiciousness.

3. Implementation Phases

Phase 1: The LibCST Mutation Engine

Use `cst.CSTTransformer` to traverse the concrete syntax tree and apply Machine Learning-specific mutation operators.

- **Activation Operator Replacement (AOR):** Swap `nn.ReLU` with `nn.Sigmoid`.

- **Tensor Shape Mutation (TSM):** Alter `.reshape(batch, -1)` to `.reshape(-1, batch)`.
- **Math Operator Replacement (MOR):** Swap `+` and `-` in custom loss functions. Save each mutant to a temporary directory with a ledger tracking `mutant_id`, `original_line_number`, and `mutation_type`.

Phase 2: Ground Truth Generation

Iterate over the temporary mutant files. Use `subprocess.run(["pytest", "mutant_file.py"], capture_output=True)`.

- If the test suite fails (exit code `!= 0`), label the mutant 1 (Killed).
- If the test suite passes (exit code `== 0`), label the mutant 0 (Survived).

Phase 3: Feature Engineering

For each mutant, generate a structured feature vector:

1. `mutation_type`: One-hot encoded (AOR, TSM, MOR).
2. `ast_depth`: Integer representing how deep the mutated node is in the CST.
3. `parent_node_type`: Categorical label of the surrounding logic (e.g., For, If, FunctionDef).
4. `is_covered`: Binary flag indicating if the original line was executed by a failing test (extracted via `coverage.py`).

Phase 4: Model Training & Fault Localization

1. Train a `RandomForestClassifier` on the feature dataset mapping to the Killed/Survived labels.
2. For inference on a new bug, generate the mutants, extract their features, and predict their probability of being killed.
3. Calculate the suspiciousness score for each line s using the Metallaxis formula:

$$\text{Suspiciousness}(s) = \frac{f_{\text{killed}}(s)}{\sqrt{F \times (f_{\text{killed}}(s) + p_{\text{killed}}(s))}}$$
(Where F is the total number of failing tests, $f_{\text{killed}}(s)$ is the number of failing tests predicted to kill mutants on line s , and $p_{\text{killed}}(s)$ is the number of passing tests predicted to kill mutants on line s .)

4. Proposed File Structure

Plaintext

```
ml_pmt_project/
├── target_model/
│   ├── model.py      # The PINN or Markowitz optimization script
│   └── test_model.py  # pytest suite with strict mathematical oracles
├── mutation_engine/
│   └── mutators.py    # LibCST CSTTransformer classes for AOR, TSM, MOR
```

```

|   └── generator.py    # Applies mutators and writes mutant files to /tmp
|   └── execution/
|       └── runner.py    # Subprocess wrapper for pytest ground-truth generation
|   └── features/
|       └── extractor.py  # AST depth and coverage.py feature extraction
|   └── model/
|       ├── train.py     # RandomForest training and evaluation
|       └── localize.py   # Inference pipeline and Metallaxis math implementation
|   └── data/             # Generated mutant ledgers and CSV feature datasets
|   └── requirements.txt

```

5. Agent Action Plan (Execution Instructions)

To the AI Coding Agent: Proceed step-by-step. Do not move to the next step until the current one is fully tested and verified.

1. **Step 1:** Initialize the project structure and create a mock model.py (a basic Mean-Variance Optimization pipeline using numpy) and test_model.py to serve as our target. Ensure the tests pass.
2. **Step 2:** Implement mutation_engine/mutators.py. Create a LibCST transformer that successfully targets and modifies mathematical operators and matrix/tensor shape calls. Write a test to ensure the CST serializes back to valid Python code.
3. **Step 3:** Implement the generator.py and runner.py scripts. The generator must create a ledger (mutants.json) mapping the generated files to their modified line numbers. The runner must execute pytest on all of them and append the ground-truth label.
4. **Step 4:** Build extractor.py to parse the original code, determine AST depth for specific lines, and format the final dataset into a Pandas DataFrame.
5. **Step 5:** Implement the RandomForestClassifier in train.py. Train it on the generated DataFrame.
6. **Step 6:** Implement localize.py. Inject a deliberate fault into model.py, run the prediction pipeline (bypassing the test runner), apply the Metallaxis formula, and output the ranked list of suspicious lines.